```
//App.vue

<template>
  <div class="container">
    <div class="max-w-5xl mx-auto bg-white p-6 rounded-lg shadow">
      <h1 class="text-3xl font-bold mb-8 text-center text-blue-700">Générateur de
Fiche Formation</h1>
      <form @submit.prevent>
      <TabView v-model:activeIndex="activeTab">
        <TabPanel header="Partie principale">
          <template #default>
            <div v-for="field in config.main" :key="field.id" class="mb-6" v-
if="stockage.main[field.id]">


              <div class="flex items-center mb-2">
                <input
                  type="checkbox"
                  v-model="stockage.main[field.id].hidden"
                  class="mr-2"
                  :id="'hide_' + field.id"
                />
                <label :for="'hide_' + field.id" class="text-sm">{{ field.label
}}</label>
              </div>

              <component v-if="stockage.main[field.id].hidden"
                        :is="getComponent(field.type)"
                        :field="field"
                        :zone="'main'"
                        v-model="stockage.main[field.id].value"
                        />
            </div>
          </template>
        </TabPanel>

        <TabPanel header="Informations complémentaires">
          <template #default>
            <div v-for="field in config.main" :key="field.id" class="mb-6" v-
if="stockage.main[field.id]">

              <div class="flex items-center mb-2">
                <input
                type="checkbox"
                v-model="stockage.aside[field.id].hidden"
                class="mr-2"
                :id="'hide_' + field.id"
                />
                <label :for="'hide_' + field.id" class="text-sm">{{ field.label
}}</label>
              </div>
```

```
              <component v-if="stockage.aside[field.id].hidden"
                         :is="getComponent(field.type)"
                         :field="field"
                         :zone="'aside'"
                         v-model="stockage.aside[field.id].value"
              />
            </div>
          </template>
        </TabPanel>
        <TabPanel header="Génération">
          <div class="p-4 bg-gray-100 rounded-lg">
            <h2 class="text-xl font-semibold mb-4">Génération du HTML</h2>
            <p class="mb-2">Sélectionnez les champs à afficher dans la partie
principale et les informations complémentaires.</p>
            <p class="mb-2">Cliquez sur le bouton "Générer le HTML" pour voir le
résultat.</p>
            <p class="text-sm text-gray-600">Note : Les champs masqués ne seront
pas inclus dans le HTML généré.</p>
          </div>
          <!--Bouton d'ajout dans le template-->
          <button @click="afficherHTML" class="mt-6 bg-blue-600 hover:bg-blue-700
text-white font-bold py-2 px-4 rounded">Générer le HTML</button>

            <div class="mt-6">
              <HtmlCodePreview title="Main" :code="htmlGenere[0]" />
              <button @click="copier(htmlGenere[0])" class="mt-6 bg-blue-600
hover:bg-blue-700 text-white font-bold py-2 px-4 rounded">
                Copier le HTML
              </button>
            </div>
            <div class="mt-6">
              <HtmlCodePreview title="Aside" :code="htmlGenere[1]" />
              <button @click="copier(htmlGenere[1])" class="mt-6 bg-blue-600
hover:bg-blue-700 text-white font-bold py-2 px-4 rounded">
                Copier le HTML
              </button>
            </div>
        </TabPanel>
        <TabPanel header="Configuration">
  <div class="p-4 bg-gray-100 rounded-lg space-y-6">

    <div class="flex items-center space-x-2">
      <span class="text-sm font-medium">Configuration active :</span>
      <span class="px-3 py-1 bg-white border rounded">{{ configName || 'Aucune'
}}</span>
    </div>



    <div class="space-y-2">
      <label for="configSelect" class="text-sm font-medium">Changer :</label>
      <Dropdown
```

```
        id="configSelect"
        v-model="selectedConfig"
        :options="savedConfigs"
        placeholder="Sélectionnez une configuration"
        class="w-full"
      />
      <button @click="chargerConfig(selectedConfig)" class="bg-blue-600 hover:bg-
blue-700 text-white py-2 px-4 rounded">Charger</button>
    </div>


    <div class="space-y-1">
      <label class="text-sm font-medium">Nom de la formation :</label>
      <input v-model="configName" type="text" class="w-full px-3 py-2 border
border-gray-300 rounded" />
    </div>


    <div>
      <h3 class="text-md font-semibold mb-2">Labels des champs :</h3>
      <div class="grid grid-cols-1 sm:grid-cols-2 gap-4">
        <div v-for="field in [...config.main, ...config.aside]" :key="field.id"
class="space-y-1">
          <label class="text-xs text-gray-600">ID : {{ field.id }}</label>
          <input v-model="field.label" class="w-full px-3 py-2 border rounded" />
        </div>
      </div>
    </div>


    <div class="flex gap-4">
      <button
        @click="sauvegarderConfig(configName)"
        class="bg-green-600 hover:bg-green-700 text-white py-2 px-4 rounded">
        Mettre à jour
      </button>
    </div>
  </div>
</TabPanel>


      </TabView>
      </form>
    </div>
  </div>
</template>

<script setup>

const activeTab = ref(0);
import Editor from '@toast-ui/editor';
```

```
import { ref, reactive, watch } from 'vue';
import TabView from 'primevue/tabview';
import TabPanel from 'primevue/tabpanel';
import HtmlCodePreview from './components/HtmlCodePreview.vue';
import '@toast-ui/editor/dist/toastui-editor.css';
import config from './config/formConfig.js';

const stockage = reactive({ main: {}, aside: {} });

const converter = {
  editor: null,
  init() {
    const div = document.createElement('div');
    div.style.display = 'none';
    document.body.appendChild(div);
    this.editor = new Editor({
      el: div,
      initialEditType: 'markdown',
      previewStyle: 'vertical',
      height: '0px',
    });
  },
  toHTML(markdown) {
    this.editor.setMarkdown(markdown);
    return this.editor.getHTML();
  },
};

for (const zone in config) {
  config[zone].forEach(field => {
    stockage[zone][field.id] = {
      value: field.default ?? "",
      hidden: true
      };
  });
}

function getComponent(type) {
  return {
    inputText: 'input-field',
    textarea: 'textarea-field',
    markdown: 'markdown-field',
    link: 'link-field'
  }[type] || 'unknown-field';
}

// crée bloc HTML contenant champs
function genererHTML(zConfig,zData) {

  converter.init();
  let html = '';
```

```
  for (const field of zConfig) {
    //recupere les données associés via l'ID
    const data = zData[field.id];
    //si le champ est visible et a une valeur on l'affiche
    if (data.hidden && data.value) {
      //on appele fonction genererChampHTML() pour obtenir le HTML du champ
      html += genererChampHTML(field, data.value);
    }
  }
  return html;
}

// fonction qui retourne un HTML généré pour chaque champ en fonction du type
function genererChampHTML(field, value) {
  //Condition 'switch' qui permet d'apater le format HTML en fonction du type
  switch (field.type) {
    case 'inputText':
      //pour inputText, un paragraphe simple, value est le texte saisi par
l'utilisateur
      return `<p><strong>${field.label} :</strong> ${value}</p>`;
    case 'textarea': {
        const confLc = {
          ul: { pre: "ul", it: "li" },
          ol: { pre: "ol", it: "li" },
          p:  { pre: null, it: "p" }
            };
            const convline = confLc[field.lineConverter];
            if (!convline) {
              return `<p><strong>${field.label}
:</strong><br>${value.replace(/\n/g, '<br>')}</p>`;
            }
            const lignes =
value.split('\n').map(v=>`<${convline.it}>${v}</${convline.it}>`);

            const contenu = lignes.join('');

            const labelHtml = `<p><strong>${field.label} :</strong></p>`;

            return convline.pre
              ? `${labelHtml}<${convline.pre}>${contenu}</${convline.pre}>`
              : `${labelHtml}${contenu}`;
            }
    case 'markdown':
      // il faut parser le markdown
      return `<h2>${field.label}</h2> ${converter.toHTML(value || '')}`;
    case 'link':
      // un paragraphe oùést crée un lien cliquable
      return `<p class="formation-actions"><a href="${value.url}" target="_blank"
class="border-link">${value.label}</a></p>`;
    default:
```

```
      //Cas par défaut si le type est inconnu, suis le contenu de la boucle pour le
type des champs
      return `<p><strong>${field.label} :</strong> ${value}</p>`
  }
}

//  pour stocker le HTML généré
const htmlGenere = ref(['', '']);

// Fonction pour afficher ce HTML dans la page
function afficherHTML() {
  htmlGenere.value[0] = genererHTML(config.main, stockage.main);
  htmlGenere.value[1] = genererHTML(config.aside, stockage.aside);
}

//fonction de copie
function copier(contenu) {
  navigator.clipboard.writeText(contenu);
}

const localStorage_key = 'formConfigSaved';

function sauvegarderConfig(nom = 'default') {
  const sauvegarde = {
    main: JSON.parse(JSON.stringify(stockage.main || {})),
    aside: JSON.parse(JSON.stringify(stockage.aside || {})),
    configName: nom
  };
  localStorage.setItem(`${localStorage_key}_${nom}`, JSON.stringify(sauvegarde));
  configName.value = nom;
  updateSavedConfigsList();
  alert("Configuration sauvegardée !");
}

function chargerConfig(nom = 'default') {
  const sauvegarde = localStorage.getItem(`${localStorage_key}_${nom}`);
  if (!sauvegarde) {
    alert("Aucune configuration trouvée.");
    return;
  }

  const parsed = JSON.parse(sauvegarde);
  Object.assign(stockage.main, parsed.main);
  Object.assign(stockage.aside, parsed.aside);
  configName.value = parsed.configName || nom;
  alert("Configuration chargée !");
}


const configName = ref('');
```

```
const selectedConfig = ref('');
const savedConfigs = ref([]);

function updateSavedConfigsList() {
  const keys = Object.keys(localStorage)
    .filter(k => k.startsWith(localStorage_key + '_'))
    .map(k => k.replace(localStorage_key + '_', ''));
  savedConfigs.value = keys;
}

onMounted(updateSavedConfigsList);

import configData from './config/formConfig.js';
const config = reactive({
  main: configData.main,
  aside: configData.aside
});


</script>



//main.js

//dans terminal en bas:--> npm run dev

import { createApp } from 'vue';
import PrimeVue from 'primevue/config';
import Dropdown from 'primevue/dropdown';

import App from './App.vue';

import 'primevue/resources/themes/saga-blue/theme.css';
import 'primevue/resources/primevue.min.css';
import 'primeicons/primeicons.css';
import '../main.css';

const app = createApp(App);

app.use(PrimeVue);
app.component('Dropdown', Dropdown);

app.mount('#app');


import InputField from './components/InputField.vue';
import TextareaField from './components/TextareaField.vue';
import MarkdownField from './components/MarkdownField.vue';
import LinkField from './components/LinkField.vue';
import UnknownField from './components/UnknownField.vue';
import HtmlCodePreview from "./components/HtmlCodePreview.vue";
```

```
app.component('input-field', InputField);
app.component('textarea-field', TextareaField);
app.component('markdown-field', MarkdownField);
app.component('link-field', LinkField);
app.component('unknown-field', UnknownField);
app.component('HtmlCodePreview',HtmlCodePreview);


import TabView from 'primevue/tabview';
import TabPanel from 'primevue/tabpanel';

app.component('TabView', TabView);
app.component('TabPanel', TabPanel);

//formConfig.js

export default {
  main: [
    { id: "objectifs", label: "Objectifs", type: "markdown", default: "" },
    {
      id: "ficheUrl",
      label: "Bouton vers la fiche formation",
      type: "link",
      default: { label: "Voir la fiche formation", url: "https://caensup.fr/" }
    },
    { id: "missions", label: "Missions", type: "markdown", default: "" },
    { id: "secteurs", label: "Secteurs d'activité", type: "textarea", default: "",
lineConverter: "ul" },
    { id: "debouches", label: "Métiers et débouchés", type: "textarea", default:
"", lineConverter: "ul" },
    { id: "competences", label: "Blocs de compétences", type: "markdown", default:
"" },
    { id: "inscription", label: "Inscription", type: "markdown", default: "" },
    { id: "video", label: "Vidéo de présentation", type: "inputText", default: "" }
  ],
  aside: [
    { id: "prerequis", label: "Prérequis", type: "textarea", default: "",
lineConverter: "ol" },
    { id: "rythme", label: "Rythme", type: "textarea", default: "", lineConverter:
"ol" },
    { id: "alternance", label: "Dispositif d'alternance", type: "textarea",
default: "", lineConverter: "ol" },
    { id: "financements", label: "Autres financements", type: "textarea", default:
"", lineConverter: "ul" },
    { id: "infos", label: "Informations complémentaires", type: "textarea",
default: "", lineConverter: "ul" }
  ]
}
```